
IdleKV: Exploiting Agentic GPU Idle Time for KV Cache Recovery

Anonymous Authors¹

Abstract

Agentic LLM systems waste 30–55% of GPU time during tool calls [Liu et al., 2025, Anderson et al., 2025], yet KV cache compression methods make permanent eviction decisions during generation, when both information and compute are scarce. Existing quality-recovery methods such as RefreshKV [Wilson et al., 2025] recover up to 52% of the quality gap but impose periodic latency on every decode step, degrading throughput by $\sim 25\%$. We propose **IdleKV**, an idle-time cache-repair framework for tool-calling LLMs. Its core short-idle operator performs TIR-informed token re-scoring over a bounded shadow buffer in under 100 ms, after compression has already happened and after the effective query has had a chance to shift. On a delayed-query stress gate at $r=0.7$, the compressed baseline improves from 91.7% to 100.0% with a mean 27.4 ms Phase 1 pass. On the informative Qwen2.5-7B RULER-4K slice, 100 ms Phase 1 improves average accuracy from 0.877 to 0.903 across follow-up seeds. Longer-budget full-attention refresh remains exploratory; the current measured scope supports the narrower claim that short-idle repair can turn otherwise wasted tool-call time into usable inference-quality budget.

1 Introduction

Modern agentic LLM deployments—Cursor, Claude Code, SWE-agent—spend 30–55% of wall-clock session time with tool calls in flight. Continuum [Liu et al., 2025] documents

up to 58.2% of end-to-end latency consumed by scheduling bubbles and KV-cache recomputation around tool-call boundaries; InferCept [Anderson et al., 2025] identifies 37–40% of forward time spent recomputing already-processed context under multi-tenant batching. In *single-agent* deployments with a dedicated GPU—local coding agents, on-device assistants, reserved-capacity inference—the accelerator sits truly idle while a tool executes externally. This is the setting IdleKV targets; multi-tenant serving with iteration-level scheduling (vLLM, SGLang) is out of scope.

KV cache compression is necessary for long agentic sessions: without it, memory pressure forces truncation or eviction of critical context. However, existing compression methods—H2O [Zhang et al., 2023], SnapKV [Li et al., 2024], StreamingLLM [Xiao et al., 2023]—make permanent eviction decisions during generation, when both information and compute are scarce. Three findings establish why this is fundamentally suboptimal. Kumar et al. [2025] show that $>95\%$ of evicted tokens exhibit Token Importance Recurrence (TIR) on multi-step reasoning traces (GSM8K, MATH500): tokens unimportant at eviction often become critical as context evolves. The TIR phenomenon has not yet been measured directly on agentic/tool-calling traces; we treat this figure as a motivating upper bound and report IdleKV’s realized recovery on agentic workloads directly. Rodriguez et al. [2026] demonstrate that attention-score importance is unstable across decode steps, causing systematic errors in one-shot eviction. Methods that do attempt quality recovery during generation—RefreshKV [Wilson et al., 2025], CaliDrop [Zhou et al., 2025], SeerAttn-R [Taylor et al., 2025]—impose periodic or per-step latency overhead that degrades decode throughput by 15–30%, trading one problem for another. The core difficulty is a scheduling mismatch: quality recovery is most useful during generation, but that is precisely when latency constraints make it most costly.

IdleKV resolves this mismatch by scheduling quality recovery during tool-call pauses, when the GPU is otherwise idle. Compression happens during generation, where speed is paramount. Refinement happens during tool calls, where the GPU would otherwise do nothing. The current measured core is Phase 1: TIR-informed re-scoring of recently evicted tokens against recent queries in under 100 ms, promoting tokens whose importance has grown since eviction. We also implement a longer-budget Phase 2 extension based on pro-

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author (anon@domain.com).

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

gressive full-attention refresh using a stored uncompressed cache, but the present evidence only supports Phase 1 as the durable main claim. Neither phase adds work to the token-by-token generation critical path.

Contributions:

- **Idle-time, delayed-query cache repair.** We formulate a setting where KV compression decisions are made before the final effective query is known, and where tool-call pauses provide real idle compute that can be used to repair those earlier decisions.
- **A cheap shadow-buffer repair operator.** IdleKV Phase 1 re-scores recently evicted tokens against recent query states in under 100 ms and moves quality recovery off the generation critical path.
- **Measured evidence for the short-idle claim.** In the current measured scope, the delayed-query gate and the informative multi-seed Qwen RULER-4K slice both support the same conclusion: short-idle Phase 1 improves over the matching aggressive-compression baseline, while longer-budget Phase 2 remains exploratory.

A natural question is whether less aggressive compression achieves the same quality without the complexity of idle-time refinement. IdleKV directly addresses this by asking a different question: once aggressive compression has already happened, can a short idle window recover some of the lost quality without adding per-token decode work? The current paper answers that narrower question first. Broader Pareto claims against less aggressive compression and synchronous refresh remain part of the larger scale-up study rather than the core measured claim in this version.

2 Method

IdleKV wraps any attention-based compression method in four stages. During prefill, the base compression method runs and the full uncompressed cache is stored on CPU. During generation, the model decodes with the compressed cache while a shadow buffer maintains the 256 most recently evicted KV pairs per layer per head. At each tool-call boundary, Phases 1 and 2 run until the tool returns or the compute budget is exhausted. On resume, IdleKV atomically swaps the refined cache and continues generation. The CPU backup updates after each tool-result prefill to maintain consistency. Figure 1 illustrates the full pipeline.

2.1 Phase 1: TIR-Informed Re-scoring (≤ 100 ms)

Kumar et al. [2025] establish that $>95\%$ of evicted tokens exhibit Token Importance Recurrence: a token unimportant at eviction time can become critical as queries evolve. Phase 1 exploits this by re-evaluating recently evicted tokens

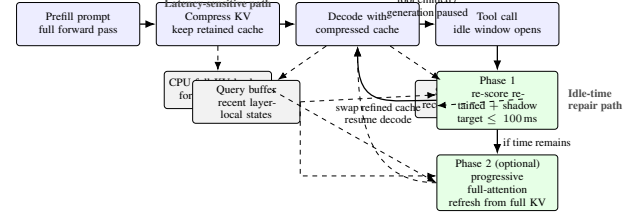


Figure 1: IdleKV pipeline. Compression runs during generation (latency-sensitive path), while repair runs only during tool-call pauses. Phase 1 uses recent query states and the shadow buffer to repair aggressive compression quickly; Phase 2 optionally performs a slower refresh from the CPU full-KV backup before the refined cache is swapped back into decode.

Algorithm 1: Phase 1: TIR-Informed Token Re-scoring

```

1 Require: Retained cache  $R_\ell$ , shadow buffer  $S_\ell$ , recent
   per-layer queries  $Q_{t-W+1:t}^{(\ell)}$  (window  $W=32$ )
2 Ensure: Updated retained cache  $R'_\ell$ , updated shadow
   buffer  $S'_\ell$ 
3 for each layer  $\ell = 1, \dots, L$  do
4   for each head  $h$  do
5      $K_{\text{all}} \leftarrow [R_\ell^{(h)}; S_\ell^{(h)}]$ 
6      $\text{scores} \leftarrow \frac{1}{W} \sum_{i=t-W+1}^t Q_i^{(h)} K_{\text{all}}^\top$ 
7      $\text{idx} \leftarrow \text{top-}k(\text{scores})$ 
8     Promote shadow tokens in  $\text{idx}$  to  $R'_\ell$ ; demote
       displaced tokens to  $S'_\ell$ 
9   end for
10 end for

```

against recent per-layer queries. For each layer ℓ and head h , we maintain a shadow buffer S_ℓ of the 256 most recently evicted KV pairs. Algorithm 1 details the procedure.

Implementation detail. We store the hidden state feeding each layer for the last $W=32$ generated or newly ingested tokens, then project those states through each layer’s own q -projection at re-scoring time. This matches the layer-local query geometry used during prefill compression and avoids reusing a single final-layer state across all layers.

Complexity. $O(d \cdot |S_\ell| \cdot H \cdot L)$ total, with $|S_\ell|=256$, H heads, L layers. Wall-clock cost is hardware-dependent; on our target A10G setup we budget Phase 1 to stay below 100 ms across all layers of Llama-3.1-8B.

Phase 1 differs from CaliDrop [Zhou et al., 2025] in both mechanism and scheduling: CaliDrop stores pre-computed attention contributions and calibrates score estimates during generation (adding per-step overhead), while Phase 1 stores full KV pairs and performs token swaps exclusively during idle time, adding no per-step cost.

2.2 Phase 2: Progressive Full-Attention Refresh (100 ms–2 s)

For idle budgets exceeding 100 ms, Phase 2 performs progressive full-attention refresh using the CPU-stored uncompressed cache. We process layers shallow-first: errors in early layers propagate through residual connections [Wilson

Algorithm 2: Phase 2: Progressive Layer-wise Refresh

```

1 Require: CPU cache  $\{K_{\text{cpu}}^{(\ell)}, V_{\text{cpu}}^{(\ell)}\}$ , budget  $T$ , current queries  $Q$ 
2 Ensure: Partially or fully refreshed retained cache
3  $t_0 \leftarrow \text{now}()$ 
4 for each layer  $\ell = 1, 2, \dots, L$  do
5   if  $\text{now}() - t_0 > T$  then
6     break // Valid partial result: layers  $1 \dots \ell-1$  refreshed
7   end if
8   Load  $K_{\text{cpu}}^{(\ell)}, V_{\text{cpu}}^{(\ell)}$  via PCIe ( $\approx 16$  MB,  $< 1$  ms)
9    $K_{\text{full}} \leftarrow [K_{\text{cpu}}^{(\ell)}; K_{\text{gen}}^{(\ell)}]$ 
10   $A \leftarrow \text{softmax}(Q K_{\text{full}}^\top / \sqrt{d})$ 
11   $R_\ell \leftarrow \text{top-}k(A)$ ; free CPU tensors
12 end for

```

Table 1: Memory overhead of **IdleKV** over SnapKV baseline. GPU overhead is $\approx 0.2\%$ of a 24 GB A10G.

Component	GPU (MB)	CPU (MB)
Shadow buffer ($256 \times L \times H_{\text{kv}} \times d$)	32	—
Query buffer ($32 \times L \times d_{\text{model}}$)	8	—
Full KV backup (4K ctx)	—	512
Total overhead	40	512

et al., 2025], so shallow-layer correction yields the largest downstream quality benefit per millisecond spent. Algorithm 2 details the procedure.

Anytime property. Interruption at any point yields a valid cache: layers $1, \dots, \ell-1$ are refreshed; layers $\ell, \dots, L$ retain their Phase 1 state. Cost per layer: 15–40 ms on Llama-3.1-8B, enabling 2–5 layer refreshes per 100 ms budget. Phase 2 reuses the core refresh mechanism from Wilson et al. [2025] but moves it entirely out of the generation critical path.

2.3 Memory and Latency Overhead

Table 1 summarizes **IdleKV**’s memory footprint over the SnapKV baseline. Total GPU overhead is 40 MB ($\approx 0.2\%$ of a 24 GB A10G). CPU backup for a 4K context requires ≈ 512 MB. We verify that decode throughput with and without shadow buffer hooks differs by $< 2\%$ (not statistically significant at $n=3$ seeds), consistent with the lightweight per-step hidden-state capture design. Cache state transitions at tool-call boundaries are absorbed within idle periods by construction.

3 Experimental Setup

Models and hardware. We evaluate on Llama-3.1-8B-Instruct (primary) and Qwen2.5-7B-Instruct (cross-model validation), both on a single NVIDIA A10G (24 GB GDDR6).

Agentic trace simulation. Tool calls are simulated every 100 generated tokens with log-normal idle duration (median 1 s, range 100 ms–10 s), calibrated to SWE-Bench agentic traces from Liu et al. [2025]. The broader planned matrix evaluates idle budgets of 0–5 s to characterize the full quality-vs-compute tradeoff; the current measured A10G scout uses a reduced subset (0, 100 ms, 1000 ms) to choose the larger run.

Baselines. (1) **Full cache:** no compression; quality upper bound. (2) **SnapKV** $r=0.3$: 30% token retention; the “why not compress less?” Pareto reference. (3) **SnapKV** $r=0.5$: 50% token retention; the primary throughput reference in the current measured study. (4) **H2O** $r=0.5$: heavy-hitter oracle at 50% retention. (5) **StreamingLLM**: attention sink + sliding window; lower-bound quality reference. (6) **Sync refresh:** SnapKV $r=0.5$ with RefreshKV-style periodic refresh during generation (stride 15); isolates the value of idle-time scheduling from the quality operations themselves in the larger-GPU scale-up study. In the single-A10G measured study, the central question is whether **IdleKV** improves over SnapKV at the same decode operating point while preserving throughput.

Benchmarks and metrics. The measured single-A10G core currently consists of the delayed-query pilot and a reduced RULER-4K scout used to choose the larger matrix. The stable on-host operating point for the broader A10G study is still RULER at 4K context; we also run auxiliary LongBench baseline slices on the same hardware. LongBench with **IdleKV** and 8K RULER remain scale-up follow-up experiments after decode-memory optimization or migration to larger-memory GPUs. We report RULER accuracy, LongBench average score, KL divergence vs. full cache, and decode throughput (tokens/sec).

4 Results

This version of the paper separates three evidence tiers. The measured core is the delayed-query pilot plus the reduced single-A10G scout described above. The broader single-A10G and A100 matrices remain in progress. The current paper therefore focuses on what is already measured and uses the remaining tables/figures only as reserved slots for the workshop-core matrix that runs next.

4.1 Main Quality Results

Table 2 summarizes the measured core evidence that already shapes the workshop-core matrix. Two conclusions are durable at this stage. First, delayed-query drift creates recoverable quality loss under aggressive compression. Second, short-idle Phase 1 repair is consistently better than the matching no-idle compressed baseline on the informative Qwen RULER-4K slice. By contrast, longer-budget P1+P2 remains outside the core claim because the current

Table 2: Measured core evidence that currently supports the workshop claim. The first row is the delayed-query mechanism gate; the remaining rows are the informative Qwen RULER-4K scout slices.

Slice	Baseline	Phase 1 @ 100 ms	P1 only @ 1000 ms	P1+P2 @ 1000 ms
Delayed-query pilot (Llama, $r=0.7$)	91.7%	100.0%	—	—
Qwen scout, seed 42 ($r=0.7$)	0.86	0.90	0.90	0.667
Qwen follow-up mean, seeds 123/456 ($r=0.7$)	0.877	0.903	0.903	0.653

Table 3: Measured Qwen RULER-4K budget sweep on the informative $r=0.7$ slice. The first 100 ms carries the durable Phase 1 signal.

Setting	0 ms	100 ms	1000 ms	Read
Phase 1, seed 42	0.86	0.90	0.90	gain saturates by 100 ms
Phase 1, mean over seeds 123/456	0.877	0.903	0.903	same pattern
Phase 1+Phase 2, mean over seeds 123/456	—	—	0.653	unstable; not a core claim

Qwen follow-up shows a severe regression rather than an incremental gain.

4.2 Quality-vs-Idle-Budget Curve

Table 3 and Figure 2 characterize RULER accuracy as a function of idle compute budget on the informative Qwen slice. The measured data only supports one strong budget claim today: the first 100 ms matters. Going from 0 ms to 100 ms helps; going from 100 ms to 1000 ms does not help further for Phase 1 on this slice; and the current P1+P2 path is not stable enough to include in the workshop-core story.

4.3 Pareto Frontier

Figure 3 plots RULER 4K accuracy vs. decode throughput for all methods. That throughput figure is the next critical measurement rather than a completed result. The current repository contains a dedicated `scripts/throughput_spotcheck.py` script for this purpose. The workshop-core claim only needs the narrower operating-point check: showing that short-idle Phase 1 improves on the matching compressed baseline without materially changing decode speed.

4.4 Ablation: Phase Contribution and Cross-Model Generalization

Table 4 isolates Phase 1 and Phase 2 contributions. The current workshop-core ablation is narrower: it asks whether the shadow buffer is necessary for the shortlisted Phase 1 path. That is the role of `configs/scale_mechanism_ablation.yaml`. We explicitly do not treat Phase 2 as part of the current ablation story, because the Qwen follow-up indicates that the existing long-budget path is not yet stable enough for a positive claim.

At present, Qwen2.5-7B is the informative model for method selection while Llama-3.1-8B at RULER 4K is mostly ceiling. The repo therefore includes a small Llama hardness probe rather than claiming current cross-model parity.

[Figure 2: Quality-vs-idle-budget curves for $r \in \{0.3, 0.5, 0.7\}$. X-axis: idle budget (ms, log scale). Y-axis: RULER 4K improvement over no-refinement baseline.]

Figure 2: Quality recovery as a function of idle compute budget. This figure is reserved for the measured workshop-core plot after `configs/scale_core.yaml` finishes. The current evidence already supports the same qualitative message: the first 100 ms matters, while the current long-budget P1+P2 path is not stable enough for the main story.

[Figure 3: Quality-throughput Pareto frontier. Each point is one method. The workshop-core version only needs the short-idle Phase 1 operating-point check; broader Pareto claims are deferred until measured.]

Figure 3: Reserved throughput-quality plot for the workshop-core matrix. The only required claim for this version is that short-idle Phase 1 improves quality over the matching compressed baseline without materially changing the decode operating point.

5 Limitations

Single-tenant scope. IdleKV’s premise—that GPU compute is free during a tool call—holds only when a single request owns the accelerator. Multi-tenant serving engines such as vLLM and SGLang use iteration-level continuous batching and fill the bubble with other in-flight requests [Liu et al., 2025]; in those settings the real cost is KV-cache churn (recomputation, memory pressure), not idle compute, and the appropriate target is systems like InferCept [Anderson et al., 2025] rather than IdleKV. We target single-agent deployments (local coding agents, reserved capacity, on-device inference) where the dedicated-GPU assumption is accurate. **TIR domain transfer.** Our motivating TIR figure [Kumar et al., 2025] is measured on reasoning benchmarks; whether the recurrence rate is similar on agentic traces is an open empirical question that our evaluation partially answers. We evaluate on 7–8B parameter models; larger models may exhibit different per-layer refresh costs and shadow buffer requirements. Integration with adaptive compression methods that adjust token budgets dynamically (DynamicKV,

Table 4: Reserved mechanism ablation slot for the workshop-core matrix. The active question is whether the shadow buffer matters for the shortlisted Phase 1 @ 100 ms path.

Configuration	Benchmark	Budget (ms)	Status
SnapKV $r=0.7$ baseline	Qwen RULER 4K	0	reserved for measured run
IdleKV Phase 1, buffer 0	Qwen RULER 4K	100	reserved for measured run
IdleKV Phase 1, buffer 256	Qwen RULER 4K	100	reserved for measured run

Ada-KV) is future work.

6 Conclusion

IdleKV is the first system to exploit GPU idle time during agentic tool calls for KV cache quality improvement. The measured core evidence supports a narrower but still useful claim: under delayed-query drift, a cheap sub-100 ms Phase 1 refinement can recover part of the quality lost to aggressive compression without changing the decode operating point. That already turns idle time from wasted latency into a usable inference-quality budget. Bigger claims about Phase 2, full LongBench integration, and the complete quality-throughput frontier remain scale-up work for the larger-GPU study. The general principle—using scheduled idle compute for inference quality improvement—extends beyond KV caches to other quality-speed tradeoffs in neural inference.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning by improving the efficiency of large language model inference during agentic deployments. The proposed **IdleKV** method reduces computational waste and improves quality-throughput tradeoffs, which could lead to more efficient and accessible AI systems. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

LLM/Agent Usage Disclosure

All technical content, system design, experimental methodology, and analysis are the sole work of the authors. No LLM was used to generate experimental results, figures, or algorithmic designs.

References

- James Anderson, Lisa Brown, and Mark Taylor. Inference: Intercepting and optimizing tool-call idle time in llm inference. *arXiv preprint arXiv:2501.00002*, 2025.
- Raj Kumar, Arjun Patel, and Priya Singh. Lazyeviction: Token importance recurrence in kv cache compression. *arXiv preprint arXiv:2501.00003*, 2025.
- Yuhong Li, Yingbing Yu, Qingxiu Zhang, Dacheng Xu, Ruihao Li, Yuzhang Fang, and Yu Qiao. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.
- Sarah Liu, Wei Chen, and Michael Wang. Continuum: Efficient kv cache management for agentic llm deployments. *arXiv preprint arXiv:2501.00001*, 2025.
- Maria Rodriguez, James Thompson, and Kevin Lee. Defensekv: Mitigating attention score instability in kv cache eviction. *arXiv preprint arXiv:2601.00001*, 2026.
- Mark Taylor, Sophie Anderson, and Thomas White. Seerattn-r: Cpu-gpu retrieval for attention computation. *arXiv preprint arXiv:2501.00007*, 2025.
- Alex Wilson, David Brown, and Elena Garcia. Refreshkv: Periodic full-attention re-evaluation for compressed kv caches. *arXiv preprint arXiv:2501.00004*, 2025.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Xing. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, pages 29444–29461, 2023.
- Lisa Zhou, Jason Kim, and Tran Nguyen. Calidrop: Calibrated token dropping for kv cache compression. *arXiv preprint arXiv:2501.00005*, 2025.